Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
Пермский национальный исследовательский политехнический университет (ПНИПУ)

Факультет: электротехнический (ЭТФ)
Направление: 09.03.04 программная инженерия
Профиль: разработка программно-информационных систем
Кафедра информационных технологий и автоматизированных систем (ИТАС)

Зав. кафедрой: д-р экон. наук, проф.
Файзрахманов Р.А. _____
«_____» _____ 2026 г.

КУРСОВАЯ РАБОТА

по дисциплине «Технологии разработки программного обеспечения»
на тему
«Разработка информационной системы учёта успеваемости студентов»

Состав курсовой работы:
1. Пояснительная записка на ____ стр.
2. Графический материал.
3. Электронный носитель с материалами курсовой работы.

Выполнил: студент группы РИС-23-1-бзу
Ситников Александр Павлович
Руководитель: Кравец В.А.
Оценка: _____

Пермь 2026

Министерство науки и высшего образования Российской Федерации
Пермский национальный исследовательский политехнический университет (ПНИПУ)

ЗАДАНИЕ

на выполнение курсовой работы бакалавра

Фамилия, имя, отчество: Ситников Александр Павлович
Группа: РИС-23-1-бзу
Начало выполнения работы: 10.10.2026
Контрольные сроки просмотра работы кафедрой: 1) _____; 2) _____; 3) _____
Дата защиты курсовой работы: _____

1. Наименование темы: «Разработка информационной системы учёта успеваемости студентов».
2. Цель курсовой работы: разработать информационную систему, автоматизирующую учёт учебных групп, занятий, оценок и посещаемости студентов, а также формирование отчётов для преподавателей и администраторов.
3. Задачи курсовой работы:

– изучить предметную область учёта успеваемости и существующие способы её автоматизации;

– сформулировать функциональные и нефункциональные требования к системе;

– спроектировать архитектуру, модель данных, API-контракт и пользовательский интерфейс;

– построить UML- и BPMN-модели автоматизируемых процессов;

– реализовать серверную и клиентскую части приложения;

– провести тестирование и подготовить демонстрационные материалы.

4. Ожидаемые результаты курсовой работы:

– работающее веб-приложение с разграничением ролей администратора, преподавателя и студента;

– электронный журнал оценок и посещаемости с отчётами и экспортом в CSV;

– комплект проектной документации: архитектурная схема, UML-диаграммы, BPMN-модели, описание API и базы данных;

– результаты автоматизированного тестирования и скриншоты основных экранов.

5. Основная литература: методические указания кафедры ИТАС, документация FastAPI, SQLite, OpenAPI, UML и BPMN, учебные пособия по проектированию информационных систем.

Руководитель курсовой работы: _____ / Кравец В.А. /
Задание принял к исполнению: _____ / Ситников А.П. /
«_____» _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1 Анализ предметной области и постановка задачи 6

1.1 Характеристика предметной области 6

1.2 Анализ существующих решений 7

1.3 Требования к системе 8

1.4 Выбор технологий и архитектурного подхода 9

2 Проектирование информационной системы 11

2.1 Архитектура системы 11

2.2 UML-модели системы 13

2.3 BPMN-модели процессов 20

2.4 Модель данных 26

2.5 API-контракт 27

2.6 Проектирование пользовательского интерфейса 29

3 Реализация и тестирование 30

3.1 Структура проекта 30

3.2 Реализация серверной части 30

3.3 Реализация клиентской части 31

3.4 Демонстрационные данные 32

3.5 Тестирование 33

3.6 Скриншоты приложения 34

3.7 Развёртывание и сопровождение 37

ЗАКЛЮЧЕНИЕ 39

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 40

ВВЕДЕНИЕ

Современная организация учебного процесса в вузе связана с обработкой большого объёма данных: учебных групп, студентов, преподавателей, дисциплин, занятий, оценок и сведений о посещаемости. Чем больше поток студентов и учебных поручений, тем выше нагрузка на преподавателей и сотрудников деканата, которые вынуждены регулярно сверять журналы, формировать ведомости и готовить отчёты. Бумажные журналы и электронные таблицы решают отдельные задачи, но не обеспечивают единого порядка хранения данных, разграничения доступа и оперативного получения согласованной информации.

Актуальность темы курсовой работы определяется необходимостью перехода от разрозненных ручных записей к единой информационной системе, которая поддерживает полный цикл учёта успеваемости: ведение справочников, планирование занятий, выставление оценок, отметку посещаемости, формирование отчётов и предоставление студенту доступа к его личным результатам. Такая система снижает трудозатраты, уменьшает число ошибок и делает данные доступными в момент принятия управленческого решения.

Объектом исследования является процесс учёта успеваемости студентов. Предметом исследования выступают методы и программные средства автоматизации этого процесса, включая проектирование архитектуры, модели данных, API и пользовательского интерфейса.

Цель курсовой работы — разработать информационную систему учёта успеваемости студентов, обеспечивающую хранение учебных данных, разграничение прав доступа, ведение электронного журнала и формирование отчётности.

Для достижения цели необходимо решить следующие задачи:

– проанализировать предметную область и существующие подходы к учёту успеваемости;

– сформулировать требования к системе и определить роли пользователей;

– выбрать стек технологий и спроектировать архитектуру приложения;

– разработать модель данных и контракт API;

– построить UML-диаграммы и BPMN-модели основных процессов;

– реализовать серверную часть, клиентское SPA и демонстрационное заполнение базы;

– провести автоматизированное тестирование и оценить полученный результат.

При выполнении работы использовались методы системного анализа, объектно-ориентированного моделирования, проектирования баз данных, API First и тестирования программного обеспечения. Проектные решения фиксировались в нотациях UML и BPMN, архитектура — в нотации Draw.io, контракт API — в формате OpenAPI 3.0.3.

Практическая значимость работы заключается в том, что созданное приложение может использоваться в учебном процессе как основа для электронного журнала успеваемости, а также как учебный пример полного цикла разработки информационной системы: от анализа требований и проектирования до реализации, тестирования и документирования.

Пояснительная записка состоит из введения, трёх разделов, заключения и списка использованных источников. В первом разделе анализируется предметная область и формулируются требования. Второй раздел посвящён архитектуре, UML- и BPMN-моделям, модели данных и API. Третий раздел описывает реализацию, тестирование и демонстрацию работы приложения.

1 Анализ предметной области и постановка задачи

1.1 Характеристика предметной области

Предметная область «учёт успеваемости» охватывает деятельность преподавателей, студентов и административного персонала вуза. Основными сущностями являются учебная группа, студент, преподаватель, дисциплина, занятие, оценка и запись о посещаемости. Студент закрепляется за группой, группа изучает набор дисциплин, для каждой дисциплины назначаются преподаватели и проводятся занятия. По итогам занятия преподаватель выставляет оценку и отмечает посещаемость, а по завершении периода формируются отчёты по группе и по отдельному студенту.

В традиционном процессе преподаватель ведёт бумажный журнал, в котором фиксирует дату занятия, тему, оценки и отсутствующих студентов. Затем сведения переносятся в ведомость или электронную таблицу, проверяются и передаются в деканат. Такой процесс обладает рядом недостатков: данные дублируются, исправления не всегда фиксируются, а доступ к сводной информации требует дополнительного времени. Студент, как правило, узнаёт результаты с задержкой и не видит полной картины по всем дисциплинам.

Автоматизация предметной области должна обеспечить единое хранилище данных, однократный ввод сведений, разграничение прав и автоматическое формирование отчётов. При этом важно сохранить привычную для преподавателя логику работы: выбрать занятие, увидеть список студентов группы, выставить оценку и отметить посещаемость. Для администратора необходимы справочники и управление занятиями, а для студента — персональная страница с оценками и средним баллом.

К системе предъявляются требования по целостности данных: оценка и запись о посещаемости должны быть уникальны для пары «студент — занятие», а удаление связанных записей не должно приводить к появлению «висящих» ссылок. Кроме того, необходимо журналировать значимые

действия пользователей, чтобы можно было восстановить историю изменений.

1.2 Анализ существующих решений

Для автоматизации учёта успеваемости применяются несколько классов решений. Первый класс — бумажные журналы и ведомости. Они просты и не требуют инфраструктуры, но не обеспечивают быстрый поиск, аналитику и совместную работу. Второй класс — электронные таблицы (Excel, LibreOffice Calc). Они позволяют считать средние значения и строить сводные таблицы, однако не содержат встроенной модели ролей, журнала действий и защиты от одновременного редактирования. Третий класс — универсальные LMS и университетские платформы. Они обладают широкой функциональностью, но могут быть избыточны для небольшой кафедры, требовать администрирования и сложной настройки под конкретный учебный процесс.

Сравнение подходов приведено в таблице 1. Оценка выполнена по критериям, важным для учебного подразделения: скорость ввода, контроль доступа, отчётность, прозрачность изменений и сложность внедрения.

Таблица 1 – Сравнение существующих подходов к учёту успеваемости

Решение	Преимущества	Ограничения
Бумажный журнал	Привычен, не требует оборудования, минимальные затраты на старт	Ручной перенос данных, нет поиска и аналитики, сложно контролировать исправления
Электронные таблицы	Гибкие формулы, сводные таблицы, знакомый интерфейс	Нет ролей и аудита, возможны конфликты версий, слабая защита данных
Универсальная LMS	Готовая платформа, широкий набор функций, централизованное хранение	Избыточность, зависимость от администратора, сложная адаптация к частному процессу
Собственная ИС	Точное соответствие процессу, роли, аудит, отчёты, простой локальный запуск	Требует разработки и сопровождения, ограничена рамками проекта

Анализ показывает, что для небольшого учебного подразделения оптимальным является собственное веб-приложение с компактным набором функций. Оно позволяет сохранить простоту ввода, добавить разграничение

ролей и автоматические отчёты, а также использовать недорогой стек технологий без отдельного сервера базы данных.

1.3 Требования к системе

Система должна поддерживать три роли: администратор, преподаватель и студент. Администратор управляет справочниками, занятиями и пользователями, преподаватель работает с журналом своих занятий, студент просматривает личную успеваемость. Разграничение прав приведено в таблице 2.

Таблица 2 – Роли пользователей и их возможности

Роль	Доступные возможности
Администратор	Все справочники, занятия, журнал любых занятий, отчёты, экспорт CSV, просмотр аудита
Преподаватель	Просмотр справочников, занятия своих дисциплин, журнал своих занятий, отчёты по группам, экспорт CSV
Студент	Просмотр главной панели и личной успеваемости по своим дисциплинам

Функциональные требования сгруппированы по бизнес-возможностям и приведены в таблице 3. Каждому требованию присвоен идентификатор, который используется при тестировании.

Таблица 3 – Функциональные требования

ID	Функциональное требование
F-01	Аутентификация по логину и паролю, завершение сеанса, проверка текущего пользователя
F-02	Ведение справочников групп, студентов, преподавателей и дисциплин
F-03	Создание, редактирование и удаление занятий с привязкой к дисциплине, группе и преподавателю
F-04	Просмотр журнала занятия со списком студентов группы
F-05	Выставление, изменение и удаление оценки за занятие
F-06	Отметка посещаемости: присутствовал, отсутствовал, опоздал
F-07	Формирование отчёта по группе: средний балл, количество оценок, пропуски, опоздания
F-08	Формирование отчёта по студенту и его личной успеваемости
F-09	Экспорт отчёта по группе в CSV
F-10	Сводная панель показателей и журналирование действий пользователей

Нефункциональные требования определяют качество решения и ограничения среды. Они приведены в таблице 4.

Таблица 4 – Нефункциональные требования

ID	Нефункциональное требование

ID	Нефункциональное требование
N-01	Безопасность: хранение паролей в виде scrypt-хеша, подписанная HttpOnly-cookie, проверка ролей на сервере
N-02	Целостность: внешние ключи, ограничения UNIQUE и CHECK, транзакционная запись оценок и посещаемости
N-03	Производительность: отклик API на локальной базе до 1 с при демонстрационном объёме данных
N-04	Переносимость: запуск на Windows/Linux с Python 3.14, без отдельного сервера СУБД
N-05	Сопровождаемость: разделение на слои, единый контракт OpenAPI, читаемые имена модулей
N-06	Тестируемость: автоматические тесты API, изолированная временная база для тестов
N-07	Удобство: адаптивный интерфейс, понятные сообщения об ошибках, навигация по ролям
N-08	Наблюдаемость: журнал audit_log и endpoint проверки работоспособности /api/health

1.4 Выбор технологий и архитектурного подхода

Для реализации выбраны Python 3.14, FastAPI, Uvicorn, SQLite, Pydantic v2, ванильный JavaScript, HTML и CSS. Такой набор позволяет создать полноценное веб-приложение без тяжёлой инфраструктуры и сосредоточиться на проектировании и качестве кода.

FastAPI обеспечивает декларативное описание маршрутов, автоматическую валидацию входных данных, генерацию OpenAPI-схемы и удобные зависимости. Uvicorn используется как ASGI-сервер. Pydantic v2 отвечает за модели запросов и ответов, а также за единообразные сообщения об ошибках валидации.

SQLite выбрана потому, что она входит в стандартную библиотеку Python и не требует установки отдельного сервера. Для учебного проекта это снижает сложность развёртывания и сопровождения. Доступ к данным реализован через модуль sqlite3 с параметризованными SQL-запросами; ORM не используется, чтобы сохранить прозрачность запросов и уменьшить число зависимостей.

Клиентская часть выполнена как SPA на ванильном JavaScript. Это позволяет обойтись без сборщика, Node.js и фреймворка, упрощает запуск и делает код доступным для изучения. Обмен данными построен на fetch и JSON, а навигация — на hash-маршрутах.

Ключевым архитектурным принципом является API First. Файл api/openapi.yaml рассматривается как первичный артефакт: по нему генерируются Pydantic-модели сервера и JavaScript-клиент. Тест test_contract.py проверяет, что фактическая схема FastAPI соответствует контракту. Это снижает риск расхождения между документацией и реализацией.

Ограничением выбранного стека является ориентация SQLite на один узел и умеренную нагрузку. Для промышленной эксплуатации с большим числом одновременных пользователей возможен переход на PostgreSQL: слой репозиториев и SQL-запросы при этом сохраняют общую структуру, но требуют адаптации драйвера и ряда запросов.

2 Проектирование информационной системы

2.1 Архитектура системы

Архитектура приложения построена по слоистому принципу. Клиентский уровень представлен браузером и SPA: HTML-разметка, CSS-стили и JavaScript-модули отвечают за вход, навигацию, таблицы, формы и отображение сообщений. Транспортный уровень использует HTTP/HTTPS, JSON и HttpOnly-cookie с подписанным токеном. Уровень приложения реализован на FastAPI: маршрутизаторы принимают запросы, зависимости проверяют сессию и роль, Pydantic валидирует данные, репозитории выполняют SQL-запросы. Уровень данных — файл SQLite с одиннадцатью таблицами.

Отдельно на схеме выделены артефакты разработки: контракт api/openapi.yaml, скрипты генерации моделей и клиента, тесты pytest, исходники диаграмм и документация. Такое выделение подчёркивает, что контракт и проектные модели являются такими же объектами сопровождения, как и программный код.

Слои взаимодействуют через ограниченный набор интерфейсов. Клиент не обращается к базе напрямую и не содержит бизнес-правил доступа; он только отображает данные и отправляет команды. Серверная часть отвечает за проверку прав, валидацию, целостность и журналирование. Такое разделение упрощает тестирование: обработчики API можно проверять без браузера, а клиентские сценарии — на фиксированных ответах сервера.

Для локального развёртывания используется один процесс Uvicorn, который отдаёт и API, и статические файлы. В более нагруженной конфигурации статику можно вынести на отдельный веб-сервер или CDN, а API разместить за reverse-proxy. Схема при этом сохраняется: меняется только точка входа и способ доставки клиентских ресурсов.

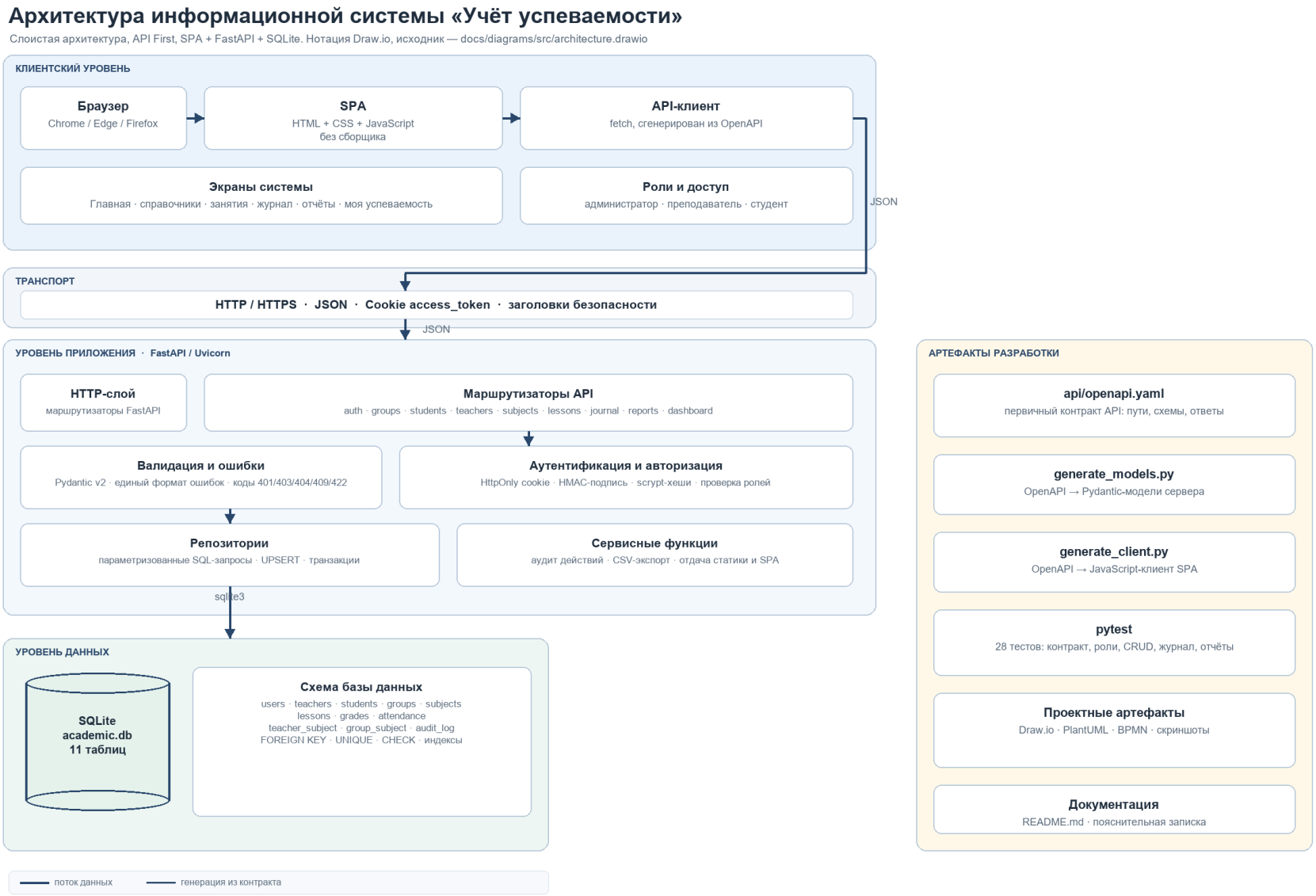


Рисунок 1 – Архитектура информационной системы «Учёт успеваемости»

Схема архитектуры показывает направление основного потока данных: браузер загружает SPA, SPA обращается к API по JSON, API проверяет сессию и права, затем через репозитории читает или изменяет данные в SQLite. Статические файлы отдаются тем же приложением FastAPI, поэтому для локального запуска достаточно одного процесса Uvicorn. Такой вариант удобен для демонстрации, но в продуктивной среде статику и API обычно разделяют reverse-proxy и отдельный сервер приложений.

Архитектура является stateless на уровне HTTP: состояние пользователя хранится в подписанном токене, а не в памяти процесса. Это упрощает перезапуск сервера и потенциальное горизонтальное масштабирование. Ограничение SQLite при этом сохраняется: несколько экземпляров приложения не должны одновременно писать в один файл без дополнительной координации.

Исходник схемы в нотации Draw.io находится в файле docs/diagrams/src/architecture.drawio, растровая версия — в docs/diagrams/architecture.png. Это позволяет при необходимости отредактировать схему и повторно вставить её в пояснительную записку.

2.2 UML-модели системы

Для описания системы использованы четыре UML-диаграммы: прецедентов, классов, последовательности и деятельности. Они дополняют друг друга: диаграмма прецедентов фиксирует функциональные границы, диаграмма классов — структуру предметной области, диаграмма последовательности — взаимодействие компонентов при сохранении оценки, диаграмма деятельности — алгоритм обработки запроса. Исходники PlantUML находятся в каталоге docs/diagrams/src.

2.2.1 Диаграмма прецедентов

Диаграмма прецедентов показывает три группы акторов и основные сценарии работы. Администратор управляет справочниками, занятиями, журналом и отчётами. Преподаватель работает с занятиями и журналом

своих дисциплин, формирует отчёты и экспортирует их. Студент авторизуется, просматривает главную панель и личную успеваемость. Все сценарии, кроме входа, выполняются в защищённой сессии.



Рисунок 2 – Диаграмма прецедентов

Диаграмма намеренно не перегружена связями include и extend: отношения между сценариями описаны в тексте, а на рисунке сохранены только ассоциации акторов с прецедентами. Это повышает читаемость и не искажает смысл модели. Прецедент «Заполнить журнал оценок и посещаемости» является центральным для системы, поскольку объединяет выставление оценки и отметку посещаемости в одной операции.

2.2.2 Диаграмма классов

Диаграмма классов отражает основные сущности предметной области и связи между ними. Пользователь имеет роль и профиль: для преподавателя создаётся запись Teacher, для студента — Student. Студент закрепляется за группой, группа изучает дисциплины, для дисциплин назначаются преподаватели и занятия. Занятие связывает дисциплину, группу и преподавателя. Оценка и посещаемость ссылаются на студента и занятие, причём для каждой пары действует ограничение уникальности.

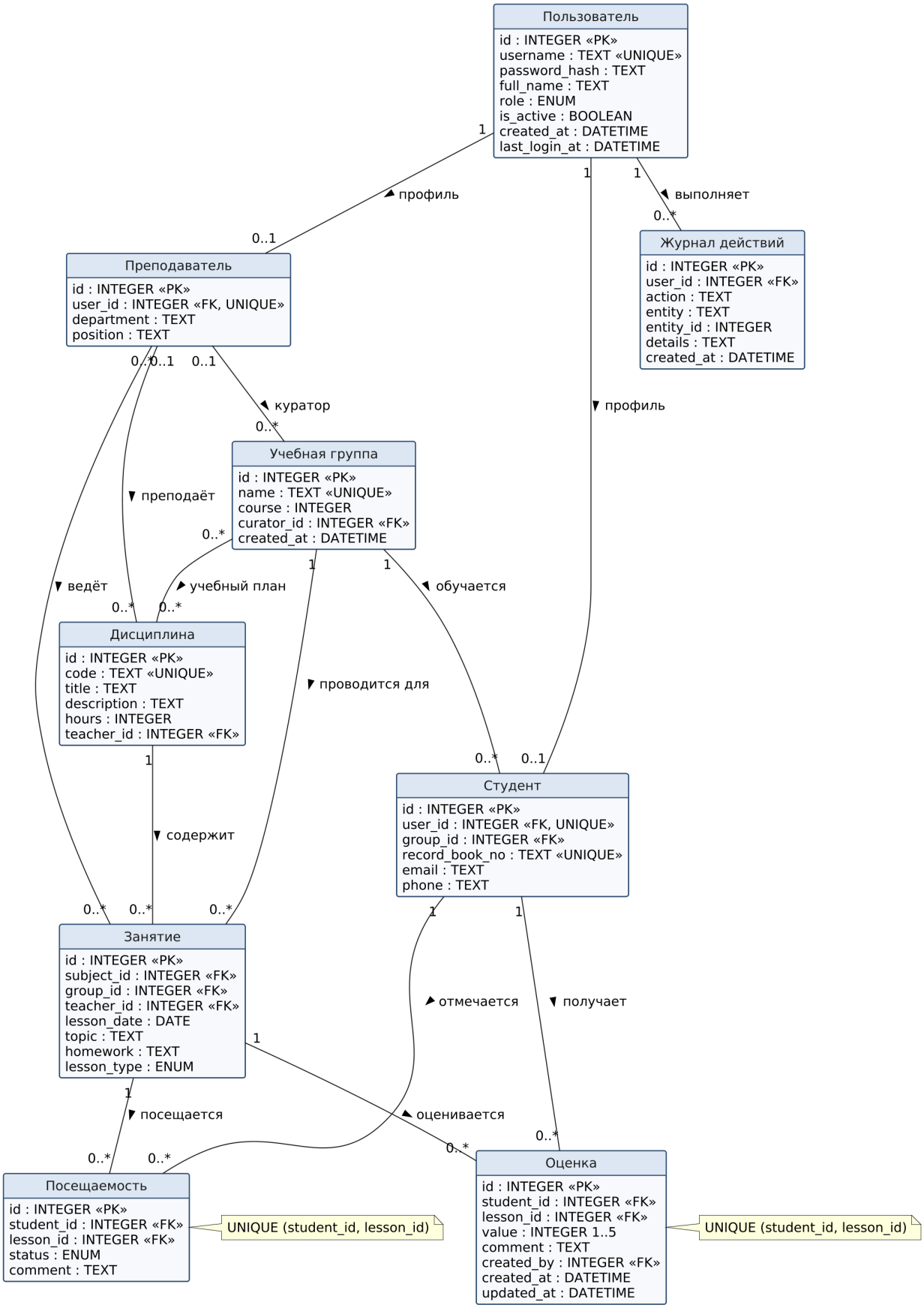


Рисунок 3 – Диаграмма классов предметной области

Связи «многие ко многим» между преподавателями и дисциплинами, а также между группами и дисциплинами реализованы через таблицы teacher_subject и group_subject. Это соответствует реляционной модели данных и позволяет гибко назначать преподавателей и формировать учебные планы. Класс AuditLog не входит в предметную область учёта, но необходим для сопровождения и расследования инцидентов.

На диаграмме указаны ключевые ограничения: уникальность логина, зачётной книжки и названия группы, допустимые значения роли и оценки, а также уникальность пары «студент — занятие» для оценок и посещаемости. Эти ограничения дублируются на уровне схемы базы данных, чтобы целостность не зависела только от кода приложения.

2.2.3 Диаграмма последовательности

Диаграмма последовательности описывает основной сценарий работы преподавателя: открытие журнала, загрузку списка студентов, изменение оценки или посещаемости и сохранение записи. Взаимодействие включает проверку сессии и роли, валидацию Pydantic, выполнение UPSERT в таблицах grades и attendance, запись действия в audit_log и возврат обновлённой записи в SPA.

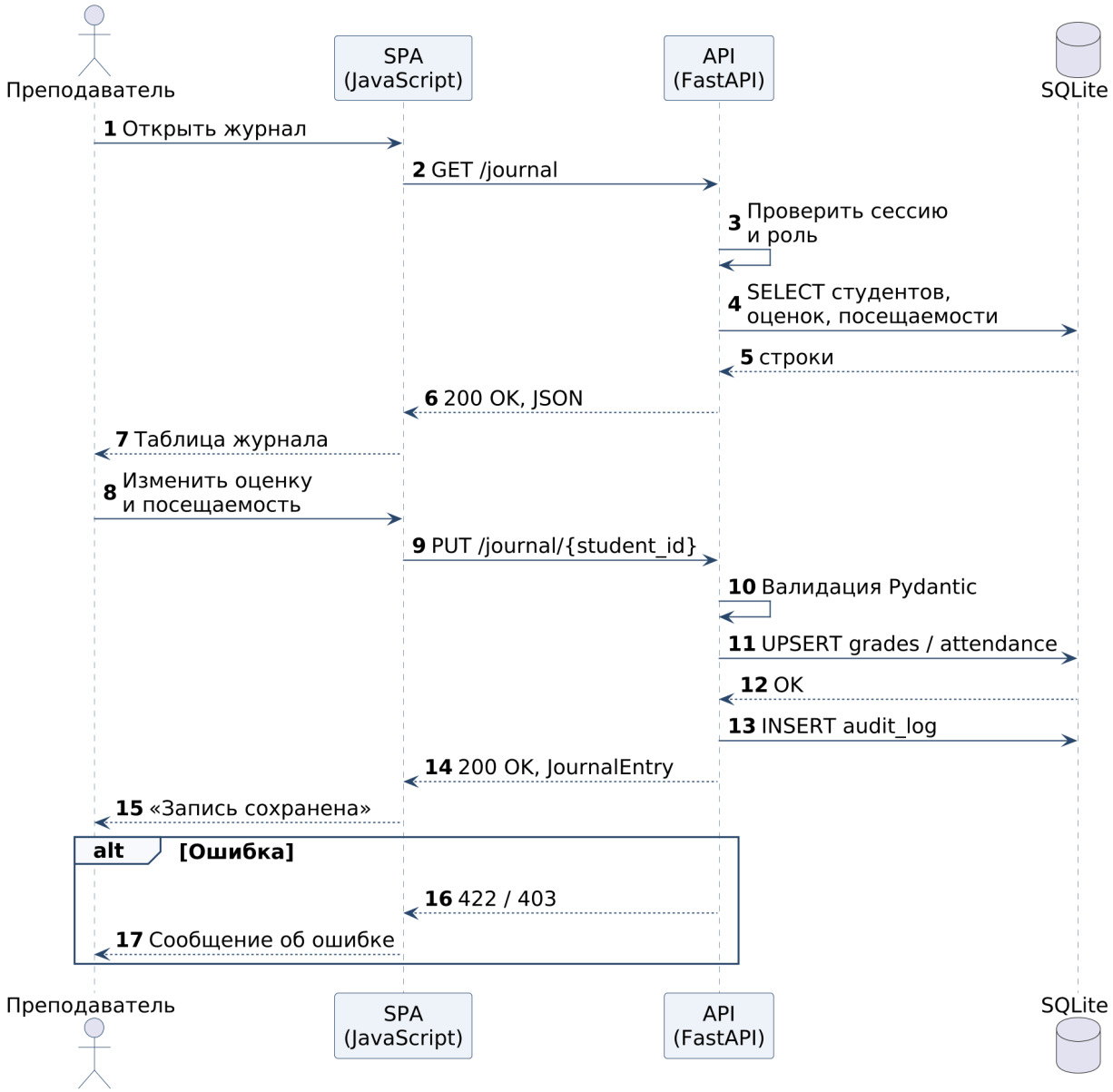


Рисунок 4 – Диаграмма последовательности сохранения записи журнала

На диаграмме отражены две ветви ошибок: некорректные данные приводят к ответу 422, отсутствие прав — к ответу 403. Сервер не доверяет данным клиента и повторно проверяет, что студент относится к группе занятия, а преподаватель имеет доступ к этому занятию. Только после этих проверок выполняется изменение базы.

Операция сохранения является транзакционной: оценка и посещаемость изменяются в рамках одного соединения. Если одна из частей завершается ошибкой, транзакция откатывается. Это важно, потому что пользователь воспринимает сохранение как единое действие, а не как два независимых изменения.

2.2.4 Диаграмма деятельности

Диаграмма деятельности формализует алгоритм обработки запроса на сохранение записи. Сначала система проверяет сессию и роль, затем загружает занятие и список студентов. После ввода данных выполняется валидация, при успехе — UPSERT и аудит, при ошибке — сообщение пользователю и возврат к редактированию. Цикл повторяется, пока преподаватель не завершит работу с журналом.

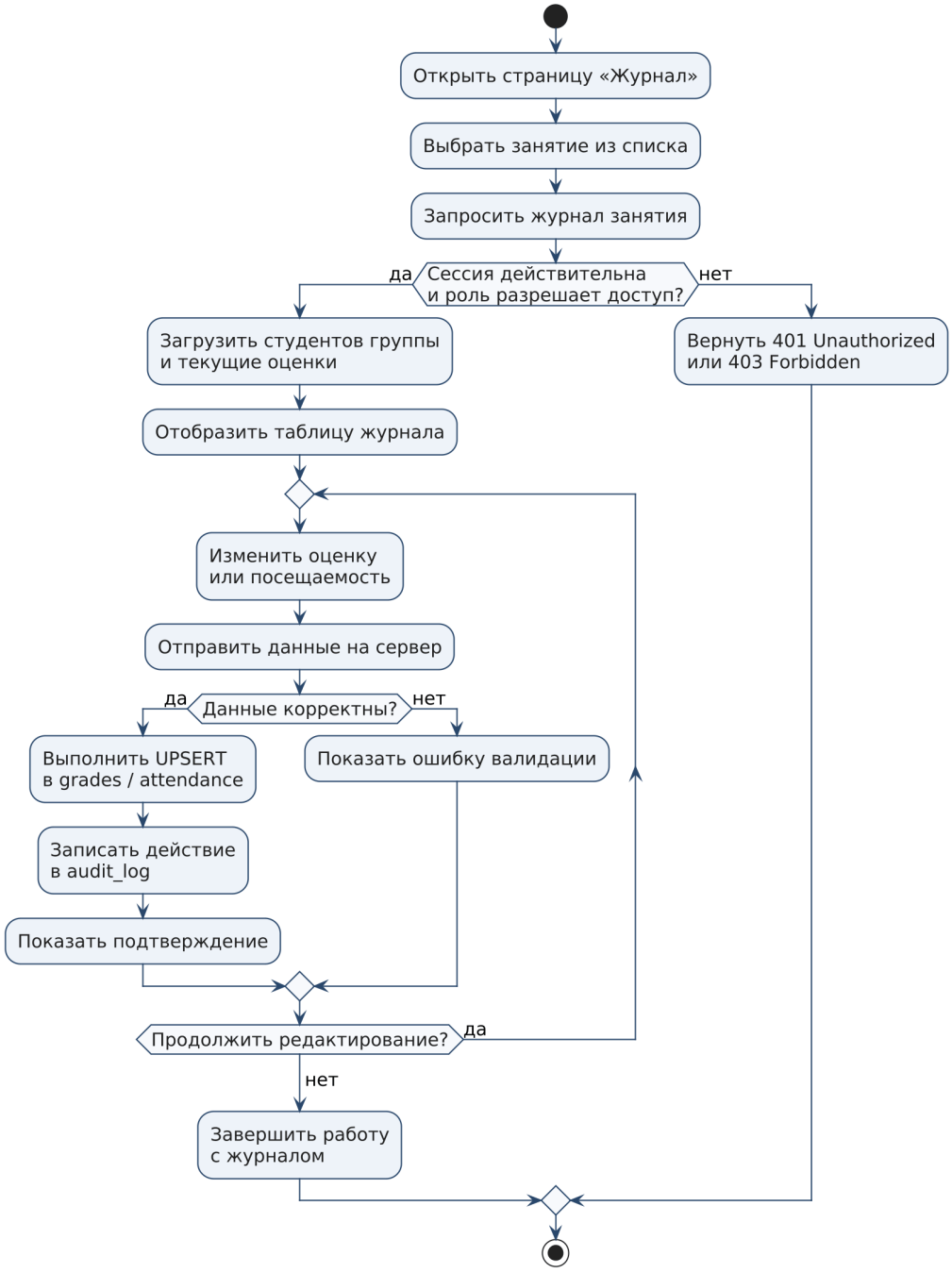


Рисунок 5 – Диаграмма деятельности при работе с журналом

Диаграмма подчёркивает, что проверка прав выполняется до обращения к данным, а валидация — до изменения базы. Разделение этих этапов позволяет возвращать пользователю точные коды ошибок и не выполнять частичные изменения. Завершение работы с журналом не требует дополнительного сохранения: каждая запись фиксируется отдельной операцией.

2.3 BPMN-модели процессов

Для описания бизнес-процессов использована нотация BPMN 2.0. В работе приведены три диаграммы: модель AS-IS (бумажный журнал), модель TO-BE (электронный журнал) и процесс формирования отчёта с экспортом в CSV. Модели используются как проектные схемы для анализа и согласования процесса; исполняемый BPMN-движок в приложении не применяется, поэтому четвёртая диаграмма, характерная для исполняемых моделей с дополнительными артефактами и обработчиками, в рамках проекта не требуется.

Такое ограничение важно зафиксировать явно: диаграммы не запускаются в Camunda, Zeebe или другом движке, а служат средством коммуникации между аналитиком, разработчиком и заказчиком. При переходе к исполняемой модели потребовалось бы определить формальные условия шлюзов, переменные процесса, обработчики задач и схему развёртывания.

2.3.1 Модель AS-IS

Модель AS-IS описывает текущий процесс ведения бумажного журнала. Преподаватель проводит занятие, заполняет журнал, передаёт его заведующему кафедрой; студент узнаёт оценку после ознакомления с журналом. Заведующий проверяет записи, при обнаружении ошибок возвращает журнал на исправление, при отсутствии ошибок подписывает и сдаёт документ в архив.



Рисунок 6 – BPMN-модель AS-IS: бумажный журнал успеваемости

Основные проблемы процесса: ручной перенос данных, задержка между выставлением оценки и её появлением у студента, отсутствие единой истории исправлений и высокий риск ошибок при формировании сводных ведомостей. Кроме того, бумажный журнал не позволяет быстро получить срез по группе или дисциплине.

С точки зрения документооборота процесс содержит несколько точек передачи ответственности. Преподаватель отвечает за первичную запись, заведующий кафедрой — за проверку и подписание, архив — за хранение. Каждая передача увеличивает время обработки и создаёт возможность утраты или искажения сведений. Исправление оценки требует повторного просмотра и не всегда оставляет заметный след в документе.

Бумажная форма плохо масштабируется при росте числа групп и дисциплин. Чтобы получить сводку по потоку, сотруднику приходится вручную переносить данные из нескольких журналов, что занимает значительное время и повышает вероятность арифметических ошибок. Отсутствие единого цифрового идентификатора студента и занятия затрудняет сопоставление записей между документами.

Поэтому целевой процесс должен не просто заменить бумагу электронной таблицей, а обеспечить единый источник данных, автоматическую проверку ограничений и мгновенное построение отчётов. Именно эти требования положены в основу модели TO-BE.

2.3.2 Модель TO-BE

Модель TO-BE описывает целевой процесс работы с электронным журналом. Преподаватель открывает журнал занятия, SPA запрашивает данные у API, сервер проверяет сессию и роль, возвращает список студентов. После ввода оценки и посещаемости SPA отправляет PUT-запрос, сервер валидирует данные, выполняет UPSERT и фиксирует действие в audit_log. Преподаватель получает подтверждение, а данные сразу становятся доступны для отчётов и личной страницы студента.

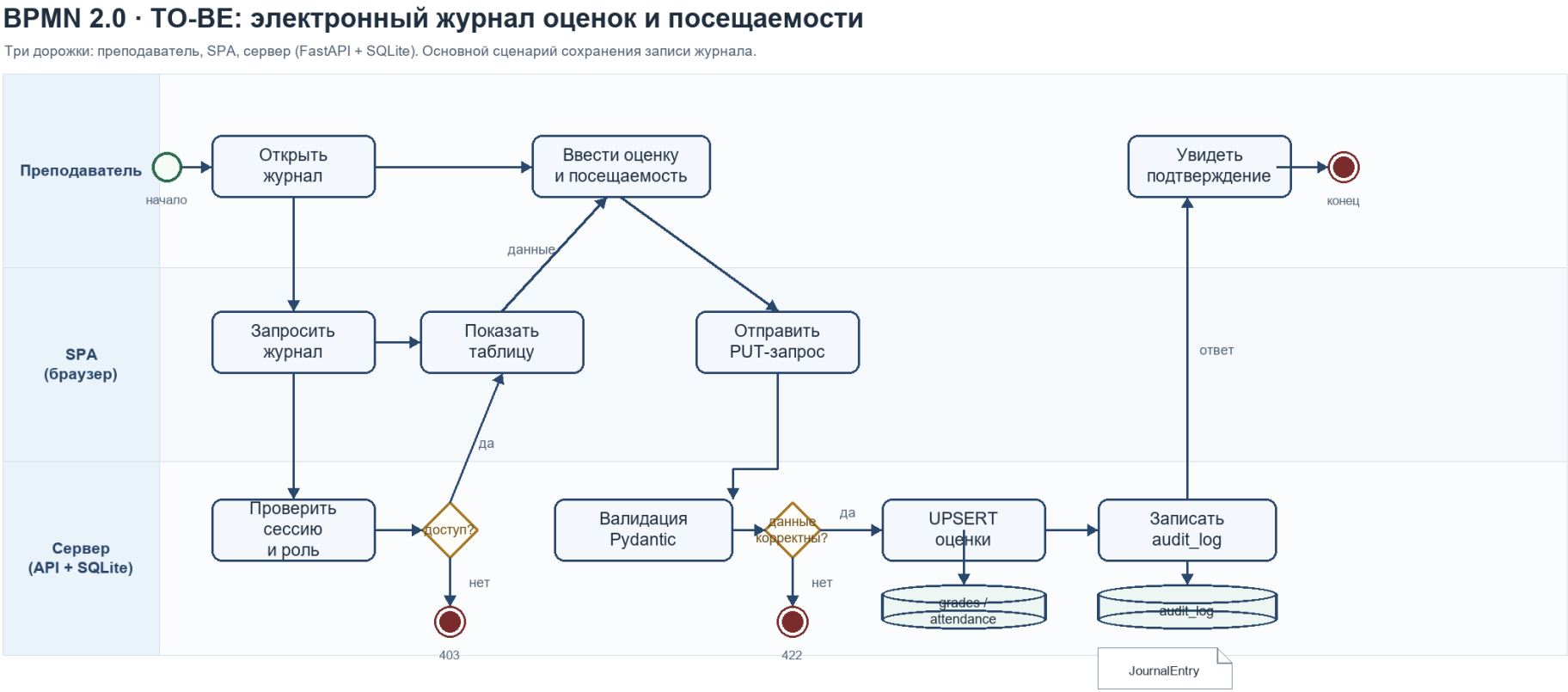


Рисунок 7 – BPMN-модель TO-BE: электронный журнал

По сравнению с AS-IS исключаются ручная передача журнала и повторный ввод данных. Оценка и посещаемость сохраняются в одной транзакции, а роль пользователя проверяется на сервере. Ошибки валидации и отсутствие прав возвращаются в виде понятных кодов 422 и 403, что упрощает обработку на клиенте.

В целевой модели каждая запись журнала имеет явные идентификаторы студента и занятия, поэтому повторное сохранение не создаёт дубликат. Операция UPSERT обеспечивает идемпотентность: повторная отправка тех же данных приводит к тому же результату. Это важно при нестабильном сетевом соединении и повторных попытках клиента.

Аудит фиксирует автора изменения и время операции. В отличие от бумажного журнала, история не теряется при исправлении: можно установить, кто и когда изменил оценку или статус посещаемости. Для учебного процесса это создаёт прозрачность и упрощает разбор спорных ситуаций.

Разграничение ролей встроено в процесс: преподаватель видит только свои занятия, администратор — все, студент — только собственные результаты. Проверка выполняется на сервере, поэтому обход интерфейса не даёт доступа к чужим данным.

2.3.3 Процесс формирования отчёта

Третья BPMN-модель описывает формирование отчёта по группе и экспорт в CSV. Преподаватель открывает раздел отчётов и выбирает группу. SPA запрашивает список групп, затем ведомость; сервер агрегирует оценки и посещаемость SQL-запросами и возвращает готовую структуру. При необходимости пользователь нажимает «Экспорт», и сервер формирует CSV-файл для скачивания.

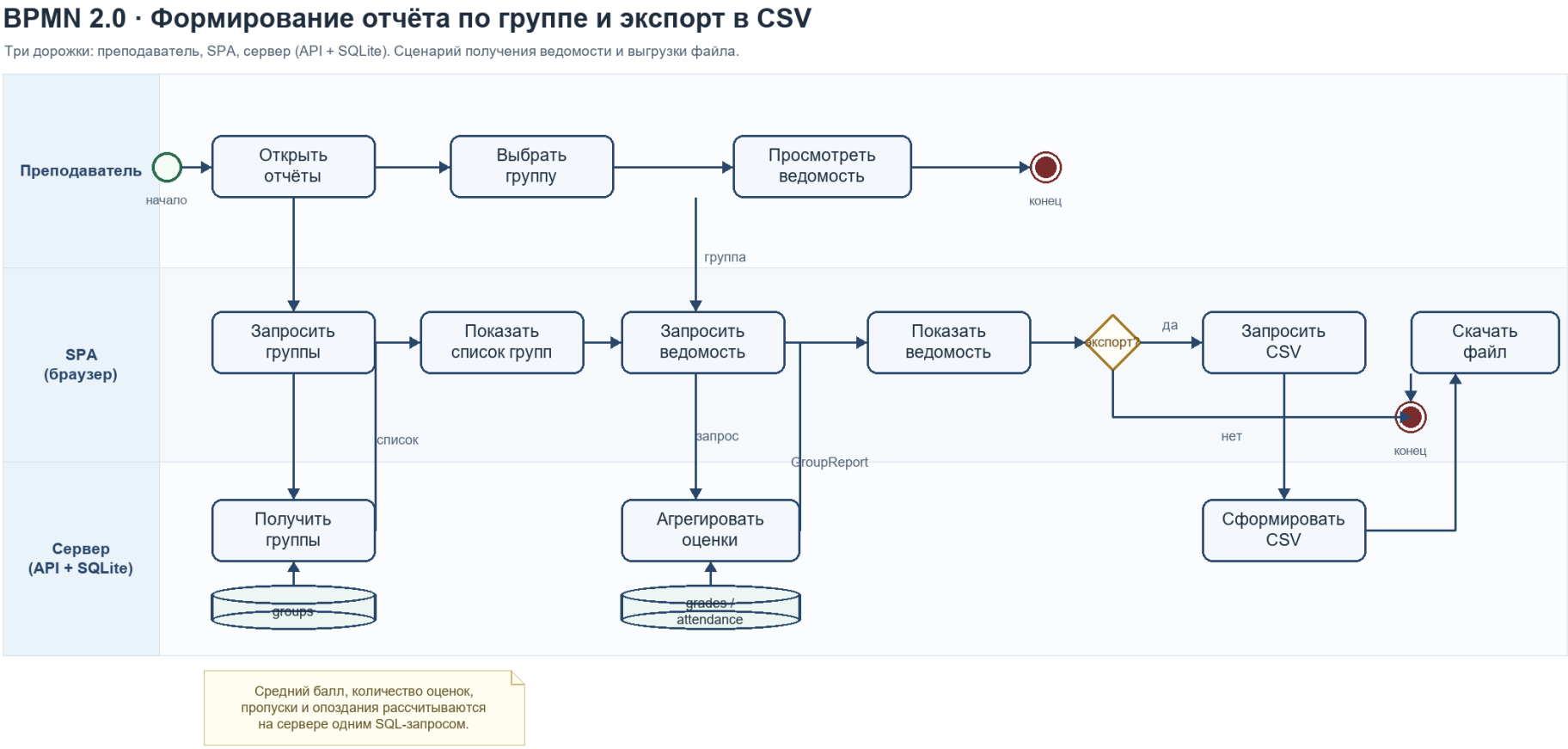


Рисунок 8 – BPMN-модель формирования отчёта и экспорта в CSV

Отчёт содержит средний балл, количество оценок, число пропусков и опозданий по каждому студенту группы. Расчёт выполняется на сервере, поэтому клиент не дублирует бизнес-логику и не зависит от объёма данных. Экспорт в CSV удобен для дальнейшей обработки в электронных таблицах и не требует установки дополнительных библиотек.

Средний балл рассчитывается как округлённое среднее значение оценок студента, а количество оценок показывает, сколько занятий уже имеют результат. Пропуски и опоздания учитываются отдельно по таблице attendance. Такая структура позволяет быстро сравнить студентов группы и выявить тех, кто нуждается в дополнительном внимании.

Экспорт в CSV формируется на сервере с корректным экранированием полей и заголовков. Файл содержит те же показатели, что и экран отчёта, поэтому может использоваться для подготовки ведомости, анализа в Excel или передачи куратору. Разделитель и кодировка выбраны так, чтобы файл корректно открывался в русскоязычной локали.

Модель отчёта не изменяет данные и не требует блокировок. Это операция только для чтения, поэтому её можно выполнять параллельно с работой преподавателей. При росте объёма данных агрегирующие запросы можно оптимизировать индексами или материализованными представлениями, не меняя контракт API.

2.4 Модель данных

Данные хранятся в реляционной базе SQLite. Схема включает одиннадцать таблиц, которые можно разделить на три группы: пользователи и профили (users, teachers, students), учебный процесс (groups, subjects, lessons, teacher_subject, group_subject) и успеваемость (grades, attendance, audit_log). Основные таблицы и их назначение приведены в таблице 5.

Таблица 5 – Основные таблицы базы данных

Таблица	Назначение	Ключевые ограничения
users	Учётные записи и роли	UNIQUE(username), CHECK(role)
teachers	Профиль преподавателя	UNIQUE(user_id), FK → users

Таблица	Назначение	Ключевые ограничения
students	Профиль студента	UNIQUE(user_id), UNIQUE(record_book_no), FK → groups
groups	Учебные группы	UNIQUE(name), CHECK(course 1..6)
subjects	Дисциплины	UNIQUE(code), FK → teachers
lessons	Занятия	FK → subjects, groups, teachers
teacher_subject	Связь преподавателей и дисциплин	Составной первичный ключ
group_subject	Учебный план группы	Составной первичный ключ
grades	Оценки за занятия	UNIQUE(student_id, lesson_id), CHECK(value 1..5)
attendance	Посещаемость	UNIQUE(student_id, lesson_id), CHECK(status)
audit_log	Журнал действий	FK → users, индекс по дате

Связи «многие ко многим» реализованы через промежуточные таблицы teacher_subject и group_subject. Для оценок и посещаемости используется уникальный составной ключ (student_id, lesson_id), что позволяет применять UPSERT: повторное сохранение не создаёт дубликат, а обновляет существующую запись. Внешние ключи включены через PRAGMA foreign_keys = ON; для части связей задано ON DELETE CASCADE или ON DELETE SET NULL в зависимости от смысла данных.

Индексы созданы по внешним ключам и часто используемым полям: дате занятия, группе студента, дисциплине. Это ускоряет выборку журнала и отчётов. Ограничения CHECK защищают от некорректных значений роли, статуса посещаемости и оценки. Пароли не хранятся в открытом виде: в поле password_hash записывается scrypt-хеш с индивидуальной солью.

Отказ от ORM объясняется небольшим размером проекта и желанием сохранить контроль над SQL. Все запросы параметризованы, поэтому значения пользовательских полей не попадают в текст SQL. Репозитории инкапсулируют запросы и возвращают словари, которые затем преобразуются в Pydantic-модели.

2.5 API-контракт

Взаимодействие клиента и сервера описано в api/openapi.yaml. Контракт содержит информацию о сервере, теги, пути, операции, схемы

запросов и ответов, коды ошибок. Основные эндпоинты приведены в таблице 6.

Таблица 6 – Основные эндпоинты API

Метод	Путь	Назначение
GET	/api/health	Проверка доступности сервера
POST	/api/auth/login	Вход и установка cookie
GET	/api/auth/me	Текущий пользователь
POST	/api/auth/logout	Завершение сеанса
GET, POST	/api/groups	Список и создание групп
PATCH, DELETE	/api/groups/{id}	Изменение и удаление группы
GET, POST	/api/students	Список и создание студентов
GET, PATCH, DELETE	/api/students/{id}	Карточка, изменение, удаление студента
GET, POST	/api/teachers	Список и создание преподавателей
GET, POST	/api/subjects	Список и создание дисциплин
GET, POST	/api/lessons	Список и создание занятий
GET, PATCH, DELETE	/api/lessons/{id}	Карточка, изменение, удаление занятия
GET	/api/lessons/{id}/journal	Журнал занятия
PUT	/api/lessons/{id}/journal/{student_id}	Сохранение оценки и посещаемости
GET	/api/reports/groups/{id}	Отчёт по группе
GET	/api/reports/groups/{id}/export	Экспорт отчёта в CSV
GET	/api/reports/students/{id}	Отчёт по студенту
GET	/api/dashboard	Сводные показатели

API следует принципам REST: ресурсы адресуются существительными, операции выражаются HTTP-методами, ответы возвращаются в JSON. Ошибки имеют единый формат с полями detail и code. Коды 401 и 403 используются для ошибок аутентификации и авторизации, 404 — для отсутствующих сущностей, 409 — для конфликтов уникальности, 422 — для ошибок валидации.

Аутентификация построена на подписанном HMAC-SHA256 токене, который передаётся в HttpOnly-cookie. Токен содержит идентификатор пользователя, роль, время выдачи и срок действия. Пароли хранятся в виде scrypt-хеша. Проверка ролей выполняется зависимостями FastAPI, поэтому обработчики не дублируют логику доступа.

Для сохранения оценки и посещаемости используется UPSERT, который вставляет новую запись или обновляет существующую. Фрагмент SQL-запроса для оценок приведён ниже.

```
INSERT INTO grades (student_id, lesson_id, value, comment, created_by)
VALUES (?, ?, ?, ?, ?)
ON CONFLICT (student_id, lesson_id) DO UPDATE SET
    value = excluded.value,
    comment = excluded.comment,
    updated_at = datetime('now');
```

Контракт API доступен в работающем приложении по адресу /api/openapi.json, а интерактивная документация Swagger UI — по адресу /api/docs. Скрипты scripts/generate_models.py и scripts/generate_client.py формируют Pydantic-модели и JavaScript-клиент, что уменьшает расхождение между сервером и SPA.

2.6 Проектирование пользовательского интерфейса

Интерфейс построен по принципу минимализма: верхняя панель с названием системы и данными пользователя, горизонтальное меню, основная область с таблицей или формой, всплывающие сообщения о результате операции. Меню формируется по роли: администратор и преподаватель видят справочники, занятия, журнал и отчёты, студент — только главную страницу и личную успеваемость.

Основные экраны спроектированы как таблицы с действиями «Изменить» и «Удалить», а также кнопкой добавления. Формы содержат обязательные поля и подсказки; ошибки выводятся рядом с полем или в общем блоке сообщения. Для журнала используется таблица, в которой по каждому студенту можно выбрать оценку и статус посещаемости, после чего сохранение выполняется отдельной кнопкой.

Цветовая схема нейтральная: тёмно-синий акцент, светло-серый фон и белые карточки. Интерфейс адаптируется к ширине окна: таблицы прокручиваются горизонтально, а меню переносится на новую строку.

3 Реализация и тестирование

3.1 Структура проекта

Проект разделён на серверную и клиентскую части, контракт API, тесты и вспомогательные скрипты. Структура каталогов приведена ниже.

```
academic performance/
├── api/openapi.yaml
├── backend/
│   ├── schema.sql
│   ├── seed.py
│   └── app/
│       ├── main.py
│       ├── config.py, db.py, deps.py, security.py, errors.py
│       ├── audit.py
│       ├── contract/models.py
│       ├── repositories/
│       └── routers/
├── frontend/
│   ├── index.html
│   ├── css/styles.css
│   └── js/app.js, js/api-client.js
├── scripts/generate_models.py, scripts/generate_client.py
├── tests/
├── requirements.txt
└── run.ps1
```

Такое разделение позволяет независимо развивать серверную и клиентскую части, а контракт API служит точкой согласования. Каталог tests содержит отдельные наборы проверок для аутентификации, ролей, CRUD, безопасности, контракта и журнала.

3.2 Реализация серверной части

Серверная часть начинается с приложения FastAPI в backend/app/main.py. При старте выполняется проверка наличия базы, при необходимости создаётся схема и загружаются демонстрационные данные. Маршрутизаторы подключаются к приложению и группируются по тегам: Auth, Groups, Students, Teachers, Subjects, Lessons, Journal, Reports, Dashboard.

Слой зависимостей отвечает за извлечение токена из cookie или заголовка Authorization, проверку подписи и срока действия, загрузку пользователя и проверку роли. Если пользователь не найден или отключён, возвращается 401. Если роль не подходит — 403. Это централизует безопасность и исключает повторную реализацию проверок в каждом обработчике.

Репозитории содержат параметризованные SQL-запросы. Соединение с SQLite создаётся на время запроса, включаются внешние ключи и busy_timeout. Транзакция завершается коммитом при успехе и откатом при исключении. Для журнала используется UPSERT, для отчётов — агрегирующие запросы с подзапросами и JOIN.

Обработчики ошибок приводят исключения к единому JSON-формату. Ошибки валидации Pydantic преобразуются в сообщение с указанием поля. Middleware добавляет заголовки безопасности: X-Content-Type-Options, X-Frame-Options, Referrer-Policy и Content-Security-Policy. Для главной страницы и статики устанавливается Cache-Control: no-cache, чтобы после обновления приложения браузер не использовал устаревшие CSS и JavaScript.

Аудит фиксирует значимые действия в таблице audit_log: пользователь, действие, сущность, идентификатор и детали. Это не полноценная система аудита безопасности, но основа для расследования изменений и анализа активности.

3.3 Реализация клиентской части

Клиентская часть расположена в каталоге frontend. Файл index.html содержит контейнеры для экрана входа и основного приложения. Стили вынесены в css/styles.css, логика — в js/app.js, обмен с API — в js/api-client.js. SPA не использует сборщик: модули подключаются как ES-модули, а версия файлов указывается в query-параметре для сброса кеша.

После успешного входа клиент получает данные пользователя, строит меню по роли и открывает главную страницу. Навигация использует hash-маршруты, поэтому состояние страницы сохраняется при перезагрузке. Для таблиц реализованы загрузка, создание, редактирование и удаление записей; для журнала — выбор занятия, отображение студентов группы, ввод оценки и статуса посещаемости.

API-клиент инкапсулирует fetch, обработку JSON и ошибок. При ответе 401 пользователь возвращается на экран входа, при 403 показывается сообщение о недостатке прав. Сообщения об успехе и ошибках выводятся в отдельном блоке и автоматически скрываются через несколько секунд.

Для удобства демонстрации на экране входа отображаются тестовые учётные записи. В продуктивной среде этот блок следует отключить. Пароли в демонстрационной базе совпадают с логинами, что допустимо только для учебного примера.

3.4 Демонстрационные данные

Скрипт backend/seed.py создаёт демонстрационную базу: 12 студентов, 3 группы, 3 преподавателя, 5 дисциплин, 39 занятий, 142 оценки и записи посещаемости. Данные распределены по группам РИС-23-1-бзу, РИС-23-2-бзу и ИВТ-24-1-бзу, что позволяет показать отчёты и журнал на реалистичном наборе.

Демонстрационные учётные записи: администратор admin/admin123, преподаватель teacher/teacher123, студент student/student123. При первом запуске приложение автоматически создаёт базу и заполняет её, если таблица users пуста. Повторный запуск не дублирует данные. Для пересоздания базы предусмотрен режим python -m backend.seed --force.

Наличие нескольких групп и дисциплин важно для проверки отчётов: средний балл, количество оценок, пропуски и опоздания рассчитываются по каждому студенту и агрегируются по группе. Демонстрационные данные

также позволяют проверить ограничения уникальности при повторном выставлении оценки за то же занятие.

3.5 Тестирование

Для проверки работоспособности использован pytest и TestClient FastAPI. Тесты работают с изолированной временной базой, поэтому не изменяют демонстрационные данные. Набор включает проверки аутентификации, разграничения ролей, CRUD справочников, журнала, отчётов, экспорта CSV, безопасности и соответствия контракту OpenAPI.

Таблица 7 – Результаты автоматизированного тестирования

Файл тестов	Что проверяется	Количество
test_auth.py	Вход, выход, текущий пользователь, health-check	7
test_contract.py	Соответствие схемы FastAPI контракту OpenAPI	4
test_crud.py	Создание, чтение, изменение и удаление справочников	4
test_roles.py	Доступ администратора, преподавателя и студента	4
test_security.py	Хеширование паролей, подпись токена, заголовки, SQL-инъекции	6
test_stage2.py	Журнал, отчёты, CSV, личная успеваемость	3

Команда запуска и результат:

```
python -m pytest -q
.......................... [100%]
28 passed, 1 warning in 4.47s
```

Все 28 тестов завершились успешно. Единственное предупреждение связано с версией httpx в окружении и не влияет на результат. Дополнительно выполнялась ручная проверка в браузере: вход под каждой ролью, навигация, создание и редактирование записей, сохранение журнала, отчёты и экспорт CSV. Скриншоты основных экранов приведены в разделе 3.6.

Ограничением тестирования является отсутствие нагрузочных испытаний и проверок на множестве одновременно работающих пользователей. Для учебного проекта этого достаточно, но перед промышленной эксплуатацией потребовались бы тесты производительности, резервного копирования и восстановления.

3.6 Скриншоты приложения

На рисунках 9–15 показаны основные экраны работающего приложения. Скриншоты сняты в браузере при демонстрационном заполнении базы.

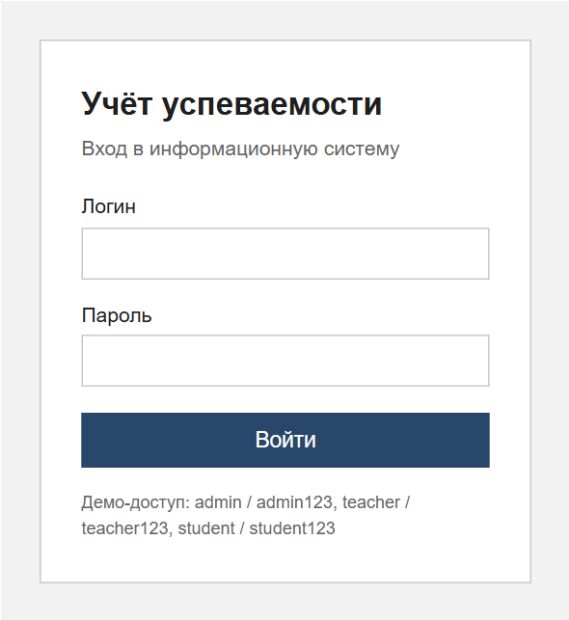


Рисунок 9 – Экран входа в систему

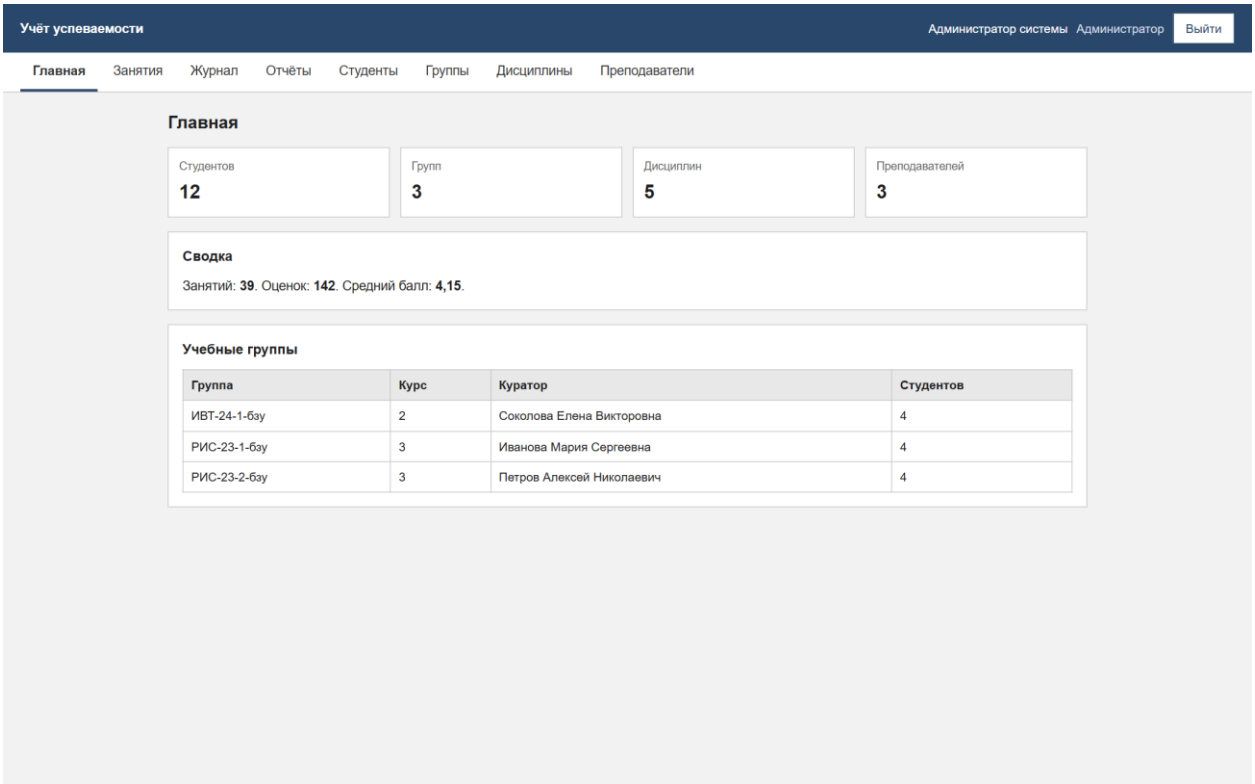


Рисунок 10 – Главная панель администратора

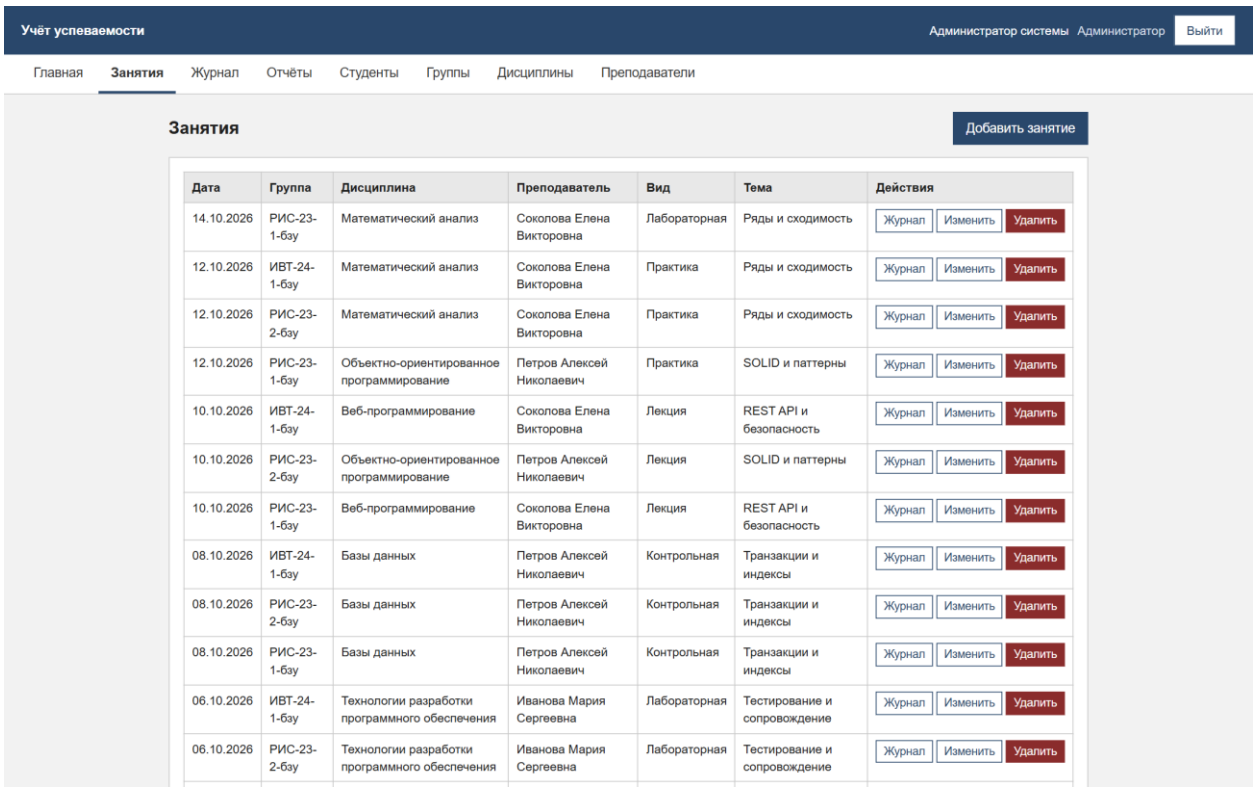


Рисунок 11 – Список занятий

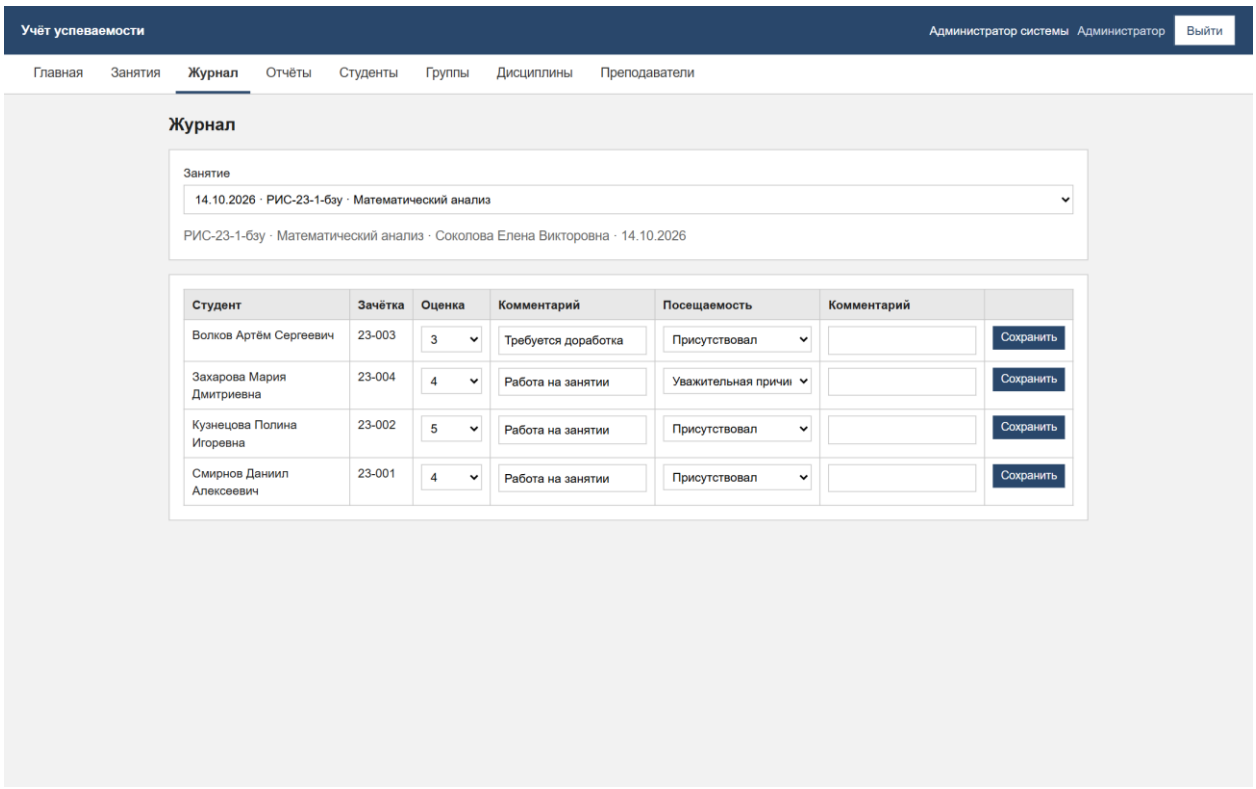


Рисунок 12 – Журнал оценок и посещаемости

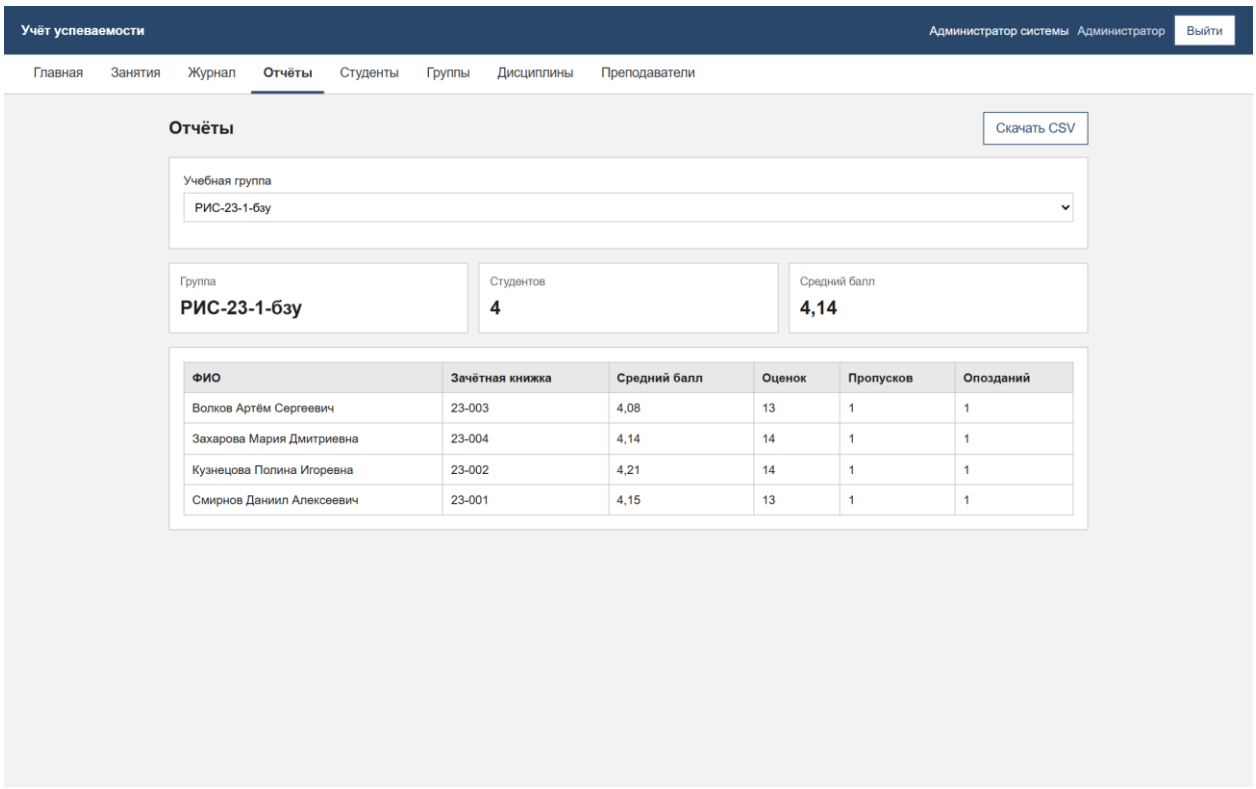


Рисунок 13 – Отчёт по группе

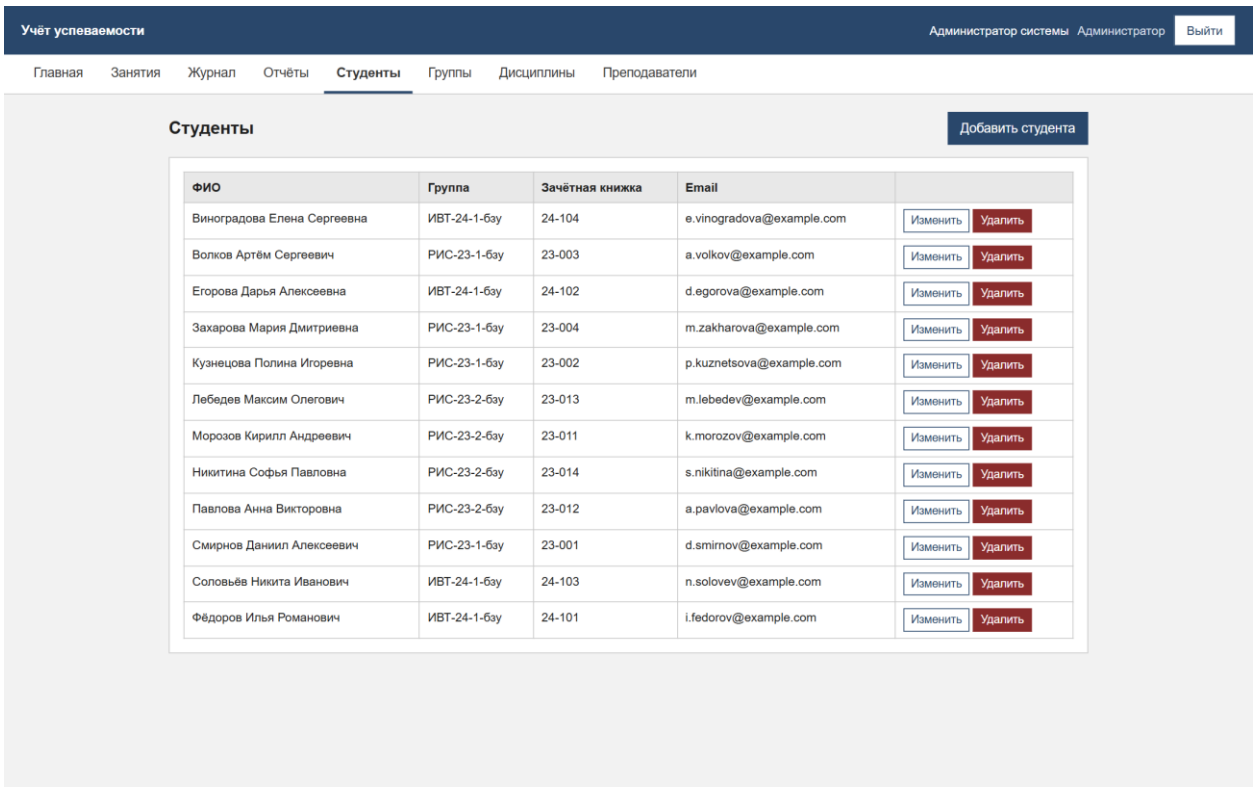


Рисунок 14 – Справочник студентов

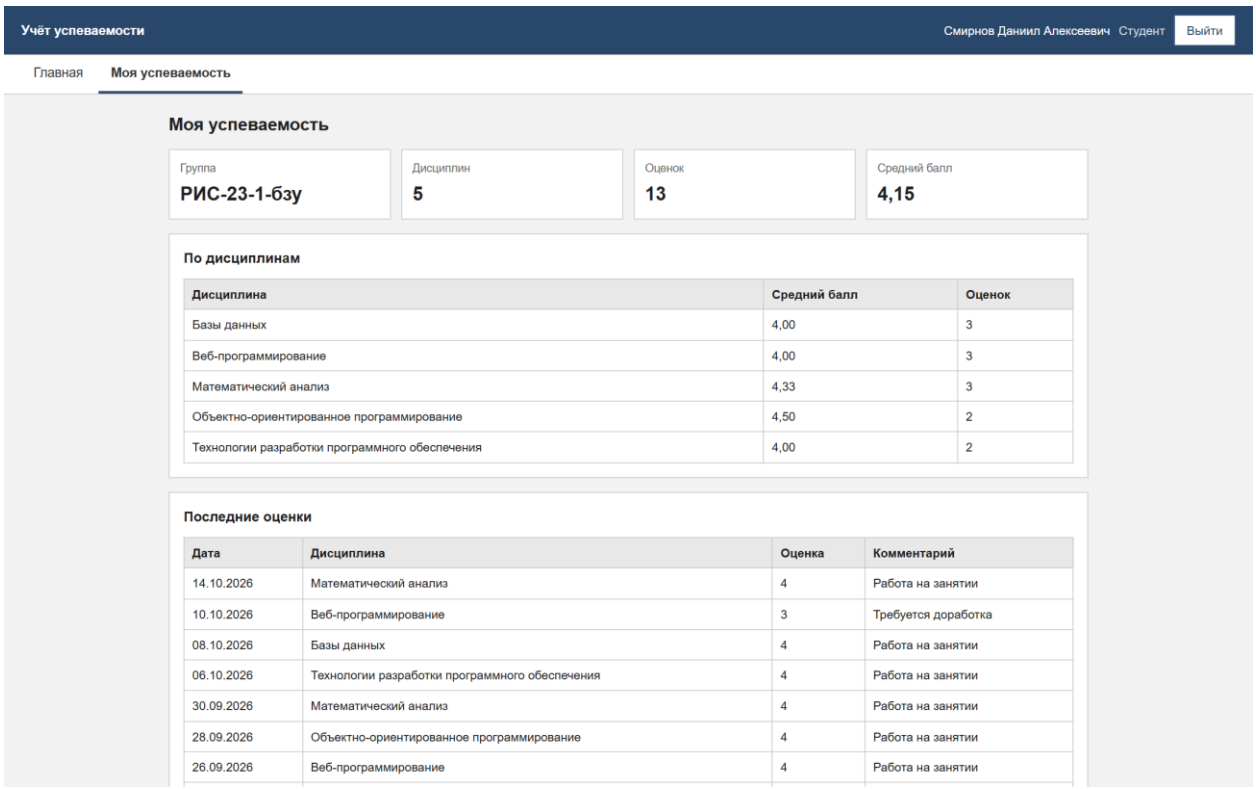


Рисунок 15 – Личная успеваемость студента

Экран входа содержит форму логина и пароля, а также подсказку с демонстрационными учётными записями. Главная панель администратора показывает количество студентов, групп, преподавателей, дисциплин и занятий, а также средний балл. Список занятий позволяет фильтровать данные, создавать и редактировать занятия. Журнал объединяет выставление оценки и отметку посещаемости в одной таблице. Отчёт по группе содержит средний балл, количество оценок, пропуски и опоздания, а также кнопку экспорта в CSV. Справочник студентов демонстрирует CRUD-операции, а личная страница студента — доступ только к своим результатам.

3.7 Развёртывание и сопровождение

Для запуска приложения достаточно установить зависимости из requirements.txt и выполнить run.ps1 или команду python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000. При первом старте создаётся файл backend/data/academic.db и загружаются демонстрационные данные. Проверка работоспособности выполняется по адресу /api/health, документация API — по адресу /api/docs.

В продуктивной среде необходимо изменить секретный ключ подписи, отключить демонстрационные подсказки, настроить HTTPS и reverse-proxy, а также регулярно создавать резервные копии файла базы. Для совместной работы нескольких серверов приложения рекомендуется перейти на PostgreSQL и вынести статику на отдельный веб-сервер. Эти изменения не требуют пересмотра архитектуры, но затрагивают конфигурацию и слой хранения.

Сопровождение проекта упрощается благодаря контракту OpenAPI и тестам: при изменении модели данных или эндпоинта необходимо обновить контракт, перегенерировать модели и клиент, затем прогнать pytest. Такой порядок поддерживает согласованность документации и кода.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы разработана информационная система учёта успеваемости студентов. Проведён анализ предметной области, сформулированы функциональные и нефункциональные требования, обоснован выбор стека технологий, спроектированы архитектура, модель данных, API-контракт и пользовательский интерфейс. Для описания системы построены четыре UML-диаграммы и три BPMN-модели, а архитектура зафиксирована в нотации Draw.io.

Реализованное приложение поддерживает аутентификацию, разграничение ролей, ведение справочников, управление занятиями, электронный журнал оценок и посещаемости, отчёты по группе и студенту, экспорт CSV и личную страницу студента. Серверная часть выполнена на FastAPI и SQLite, клиентская — как SPA на ванильном JavaScript. Контракт OpenAPI используется как первичный артефакт, по нему генерируются модели и клиент.

Автоматизированное тестирование подтвердило работоспособность основных сценариев: 28 тестов завершились успешно. Ручная проверка в браузере показала корректность навигации, форм, журнала, отчётов и разграничения доступа. Демонстрационная база содержит 12 студентов, 3 группы, 3 преподавателя, 5 дисциплин, 39 занятий и 142 записи, что достаточно для наглядной проверки отчётов.

Практическая значимость работы состоит в том, что получен готовый учебный пример полного цикла разработки информационной системы: от анализа требований и моделей до реализации, тестирования и документации. Система может быть использована как основа электронного журнала кафедры.

Направления дальнейшего развития: PostgreSQL, импорт из Excel, аналитика, интеграция с LMS и резервное копирование. Эти улучшения не меняют концепцию системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – Москва : Стандартинформ, 2009. – 6 с.

2. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. – Москва : Стандартинформ, 2010. – 27 с.

3. Буч Г. UML. Руководство пользователя / Г. Буч, Д. Рамбо, А. Якобсон. – 2-е изд. – Москва : ДМК Пресс, 2007. – 496 с.

4. Фаулер М. UML. Основы / М. Фаулер. – 3-е изд. – Санкт-Петербург : Символ-Плюс, 2004. – 192 с.

5. OMG. Business Process Model and Notation (BPMN), Version 2.0.2 : specification. – URL: https://www.omg.org/spec/BPMN/2.0.2/ (дата обращения: 05.11.2026).

6. OpenAPI Specification 3.0.3. – URL: https://spec.openapis.org/oas/v3.0.3 (дата обращения: 05.11.2026).

7. FastAPI Documentation. – URL: https://fastapi.tiangolo.com/ (дата обращения: 05.11.2026).

8. SQLite Documentation. – URL: https://www.sqlite.org/docs.html (дата обращения: 05.11.2026).

9. Python 3.14 Documentation. – URL: https://docs.python.org/3.14/ (дата обращения: 05.11.2026).

10. Pydantic Documentation. – URL: https://docs.pydantic.dev/ (дата обращения: 05.11.2026).

11. MDN Web Docs. Fetch API. – URL: https://developer.mozilla.org/ru/docs/Web/API/Fetch_API (дата обращения: 05.11.2026).

12. OWASP. Application Security Verification Standard. – URL: https://owasp.org/www-project-application-security-verification-standard/ (дата обращения: 05.11.2026).
13. PlantUML Documentation. – URL: https://plantuml.com/ru/ (дата обращения: 05.11.2026).
14. Draw.io Documentation. – URL: https://www.drawio.com/doc/ (дата обращения: 05.11.2026).
15. Методические указания по выполнению курсовой работы по дисциплине «Технологии разработки программного обеспечения». – Пермь : ПНИПУ, 2026. – 32 с.